

Lecture 19: Vision Tasks - Detection and Segmentation

One photo, four questions

Parts of this chapter adapt LMU I2DL (slds-lmu), CC BY 4.0.

The belt camera sees ten fruit

The L18 classifier grades **one** fruit per photo. But the real camera frame looks like this - a whole stall of them.

Predict: feed this photo to the L18 classifier. What comes out?

The belt camera sees ten fruit

The L18 classifier grades **one** fruit per photo. But the real camera frame looks like this - a whole stall of them.

Predict: feed this photo to the L18 classifier. What comes out?

One label for the whole image - "pomegranate", high confidence. True, and useless: *which* fruit? *where*? *how many*?

We need models whose **output has structure**: boxes, masks. That is today - and it closes the chapter.



Outline

One photo, four tasks

Object detection

Segmentation

The sorting line, completed - and the chapter close

One photo, four tasks

What changes is the *output*, not the backbone.

The task zoo

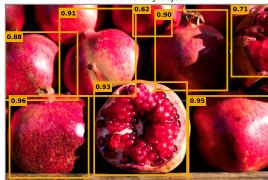
1. Classification
one label for the whole image



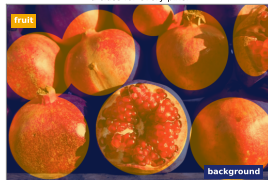
2. + Localization
the label and one box



3. Object detection
a box for every object



4. Semantic segmentation
a class for every pixel



Same photo, same conv backbone - four **output structures**: a label, a label + a box, a box *per object*, a class *per pixel*. The output structure *defines* the task. (Overlays drawn for illustration - no model ran on this photo.)

Object detection

What *and* where - a class and a box for every object in the frame.

From "what" to "what and where"

For *one* object, no new machinery is needed - just a **longer label vector**:

$$\mathbf{y} = \begin{bmatrix} b_x \\ b_y \\ b_h \\ b_w \\ c_o \\ c_{\text{pom}} \\ c_{\text{apple}} \\ c_{\text{quince}} \end{bmatrix} \begin{array}{l} \text{box center } x \\ \text{box center } y \\ \text{box height} \\ \text{box width} \\ \text{object present? (binary)} \\ \text{class, one-hot} \\ \text{class, one-hot} \\ \text{class, one-hot} \end{array}$$

Classification **and** box regression, one network: the box coordinates are just four regression outputs ([01]'s squared loss next to [11]'s log-loss).

From "what" to "what and where"

For *one* object, no new machinery is needed - just a **longer label vector**:

$$\mathbf{y} = \begin{bmatrix} b_x \\ b_y \\ b_h \\ b_w \\ c_o \\ c_{\text{pom}} \\ c_{\text{apple}} \\ c_{\text{quince}} \end{bmatrix} \quad \begin{array}{l} \text{box center } x \\ \text{box center } y \\ \text{box height} \\ \text{box width} \\ \text{object present? (binary)} \\ \text{class, one-hot} \\ \text{class, one-hot} \\ \text{class, one-hot} \end{array}$$

Classification **and** box regression, one network: the box coordinates are just four regression outputs ([01]'s squared loss next to [11]'s log-loss).

But the belt photo holds **ten** fruit. The next one, **three**.

A photo can contain **any number** of objects - but a network has a **fixed output size**. The whole field of object detection is engineering around that mismatch.

IoU: the overlap score

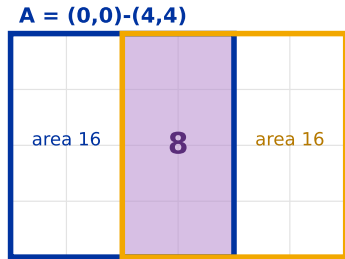
Detectors output many boxes, so we constantly ask: **how much do two boxes agree?**

$$\text{IoU}(A, B) = \frac{\text{area}(A \cap B)}{\text{area}(A \cup B)}$$

1 = identical boxes, 0 = no overlap;
in practice $\text{IoU} \geq 0.5$ usually counts
as "the same object".

Worked example: $A = (0, 0)-(4, 4)$, $B = (2, 0)-(6, 4)$. Intersection = 8, union = $16 + 16 - 8 = 24$, so $\text{IoU} = 8/24 = 1/3$.

IoU: how much do two boxes agree?

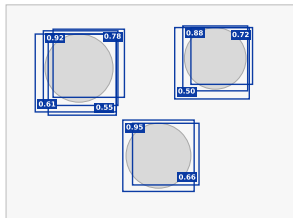


$$\begin{aligned} \text{intersection} &= 8 & \text{union} &= 16 + 16 - 8 = 24 \\ \text{IoU} &= 8 / 24 = 1/3 \end{aligned}$$

NMS: from a flood of boxes to a clean answer

A detector proposes **many overlapping candidates** per object. **Non-max suppression** cleans up. **(Click to advance.)**

the net proposes 9 boxes for 3 objects



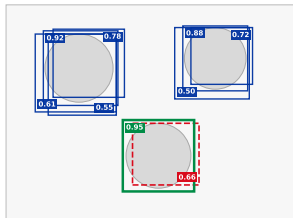
Repeat until no boxes remain: **keep** the highest-scoring box, **drop** every neighbor whose IoU with it exceeds a threshold (here 0.45). You will turn this knob yourself in HW4.

2026: the newest detectors are *trained* to emit one box per object and skip NMS - DETR, the detection transformer (Carion et al., 2020), and Ultralytics' YOLO26 (Jan 2026). The duplicates did not go away; the model learned to suppress them itself.

NMS: from a flood of boxes to a clean answer

A detector proposes **many overlapping candidates** per object. **Non-max suppression** cleans up. **(Click to advance.)**

keep the best remaining (0.95), drop neighbors with IoU > 0.45



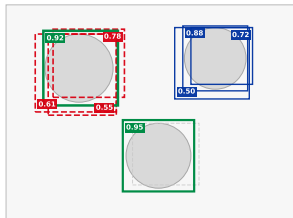
Repeat until no boxes remain: **keep** the highest-scoring box, **drop** every neighbor whose IoU with it exceeds a threshold (here 0.45). You will turn this knob yourself in HW4.

2026: the newest detectors are *trained* to emit one box per object and skip NMS - DETR, the detection transformer (Carion et al., 2020), and Ultralytics' YOLO26 (Jan 2026). The duplicates did not go away; the model learned to suppress them itself.

NMS: from a flood of boxes to a clean answer

A detector proposes **many overlapping candidates** per object. **Non-max suppression** cleans up. **(Click to advance.)**

keep the best remaining (0.92), drop neighbors with IoU > 0.45



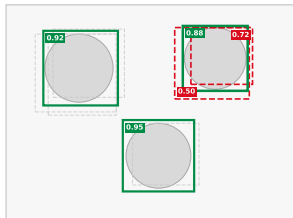
Repeat until no boxes remain: **keep** the highest-scoring box, **drop** every neighbor whose IoU with it exceeds a threshold (here 0.45). You will turn this knob yourself in HW4.

2026: the newest detectors are *trained* to emit one box per object and skip NMS - DETR, the detection transformer (Carion et al., 2020), and Ultralytics' YOLO26 (Jan 2026). The duplicates did not go away; the model learned to suppress them itself.

NMS: from a flood of boxes to a clean answer

A detector proposes **many overlapping candidates** per object. **Non-max suppression** cleans up. **(Click to advance.)**

keep the best remaining (0.88), drop neighbors with IoU > 0.45



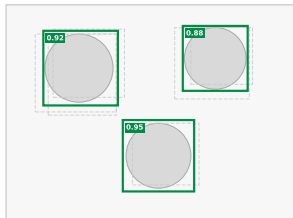
Repeat until no boxes remain: **keep** the highest-scoring box, **drop** every neighbor whose IoU with it exceeds a threshold (here 0.45). You will turn this knob yourself in HW4.

2026: the newest detectors are *trained* to emit one box per object and skip NMS - DETR, the detection transformer (Carion et al., 2020), and Ultralytics' YOLO26 (Jan 2026). The duplicates did not go away; the model learned to suppress them itself.

NMS: from a flood of boxes to a clean answer

A detector proposes **many overlapping candidates** per object. **Non-max suppression** cleans up. **(Click to advance.)**

9 candidates → 3 survivors: one box per object



Repeat until no boxes remain: **keep** the highest-scoring box, **drop** every neighbor whose IoU with it exceeds a threshold (here 0.45). You will turn this knob yourself in HW4.

2026: the newest detectors are *trained* to emit one box per object and skip NMS - DETR, the detection transformer (Carion et al., 2020), and Ultralytics' YOLO26 (Jan 2026). The duplicates did not go away; the model learned to suppress them itself.

Scoring a detector: mAP in one line

- ▶ A predicted box is **correct** if the class matches and its IoU with a true box passes a threshold (say 0.5).
- ▶ That makes every prediction a TP or FP - so **precision** and **recall** are exactly [12]'s, computed over boxes.
- ▶ **AP** = area under the precision-recall curve, per class; **mAP** = the mean over classes. Benchmarks also sweep the IoU threshold (0.5 to 0.95) and average once more.

No new metric theory: mAP is the [12] **machinery wrapped around boxes**, swept over IoU thresholds.

mAP by hand

One class, **2** true pomegranates in the photo. The detector returns 4 boxes; sort them by score and mark each **TP** (IoU ≥ 0.5 with a not-yet-matched true box) or **FP**:

rank	score		precision	recall
1	0.9	TP	$1/1 = 1.00$	$1/2 = 0.50$
2	0.8	FP	$1/2 = 0.50$	0.50
3	0.7	TP	$2/3 = 0.67$	$2/2 = 1.00$
4	0.6	FP	$2/4 = 0.50$	1.00

AP = area under the precision-recall steps
- each TP adds its recall gain times its precision:

$$AP = 0.5 \times 1.00 + 0.5 \times 0.67 = \mathbf{0.83}$$

mAP = the mean over classes: if apples score 0.60, $mAP = (0.83 + 0.60)/2 \approx 0.72$.

Ranking matters: the same four boxes with the two false alarms scored highest give $AP = 0.5$. mAP rewards finding every object *and* being most confident about the right ones. (COCO then also averages over IoU thresholds 0.50 to 0.95.)

Two families of detectors

Two-stage: first *propose* candidate regions, then classify and refine each.

- ▶ R-CNN (2014) → Fast → **Faster R-CNN** (2015)
- ▶ accurate, but two passes of work

One-stage: predict all boxes and classes in *one* forward pass.

- ▶ **YOLO** ("You Only Look Once", 2016), SSD (2016)
- ▶ real-time - video-rate detection

Two-stage bought **accuracy**, one-stage bought **speed** - and a decade of iteration has mostly closed the gap.

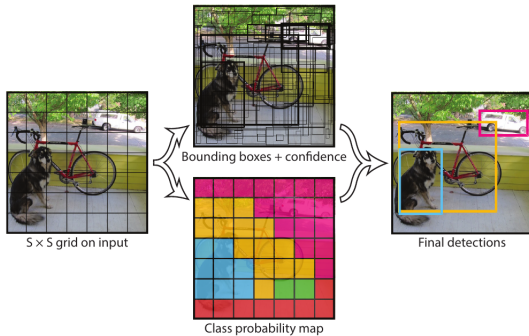
In practice, YOLO became the industry default: you `pip install` a pretrained one and fine-tune it on *your* boxes (L18's recipe). That **is** the sorting line's counting camera.

Girshick et al. 2014; Ren et al. 2015; Redmon et al. 2016; Liu et al. 2016.

How one-stage works: the YOLO grid

- ▶ Divide the image into an $S \times S$ grid (7×7 in the paper).
- ▶ **Each cell** predicts 2 candidate boxes - $(x, y, w, h, \text{confidence})$ - plus class probabilities.
- ▶ All 98 boxes come out of **one forward pass**, as one $7 \times 7 \times 30$ tensor. That is why it runs in real time.

A flood of overlapping candidates by *design* - exactly why **NMS** exists.



The paper's own diagram (Redmon et al., "You Only Look Once", 2016).

Decoding one YOLO cell, by hand

YOLO resizes every photo to 448×448 ; a 7×7 grid makes each cell 64 px wide. Take the cell in row 3, column 2 (counting from 0). One of its two boxes outputs:

$$x = 0.5, y = 0.25 \text{ (centre, relative to the cell)} \quad w = 0.2, h = 0.3 \text{ (relative to the image)} \\ \text{confidence } 0.8$$

and the cell's class head says $P(\text{pomegranate} \mid \text{object}) = 0.9$.

- ▶ centre: $((2 + 0.5) \cdot 64, (3 + 0.25) \cdot 64) = (160, 208)$ px;
- ▶ size: $0.2 \cdot 448 \approx 90$ px wide, $0.3 \cdot 448 \approx 134$ px tall;
- ▶ score for NMS: $0.8 \times 0.9 = \mathbf{0.72}$ - "is there an object here" times "which class is it".

Why 2 boxes per cell? In training, whichever of the two already overlaps the true box best is made "responsible" for it, so the two specialize in different shapes. Later YOLOs predict offsets from a few preset box shapes ("anchors") instead.

Separable convolutions: the idea

Real-time detection on a **belt camera** (or a phone) has no data center behind it. The efficiency trick that makes it fit - used by MobileNet-class backbones - splits one convolution into two cheap ones:

Two things share the name "separable": *spatially* separable factors a 2D kernel into two 1D passes (Sobel = column $\times$ row, L16); the speed win here is *depthwise* separable - factor across **channels**:

1. **Depthwise**: one small filter *per channel*, each filtering its own channel - no mixing across depth.
2. **Pointwise**: a 1×1 convolution mixes the channels back together - L17's per-pixel dense layer, doing exactly its job.

Recall (L16): a *standard* filter spans the **full input depth** - space and channels processed together. Separable says: do **space first, channels second**. Almost as expressive, far cheaper - as the next slide counts.

Separable convolutions: the count

Input $12 \times 12 \times 3$, 5×5 filters, 256 output channels $\Rightarrow$ output 8×8 . Count multiplications:

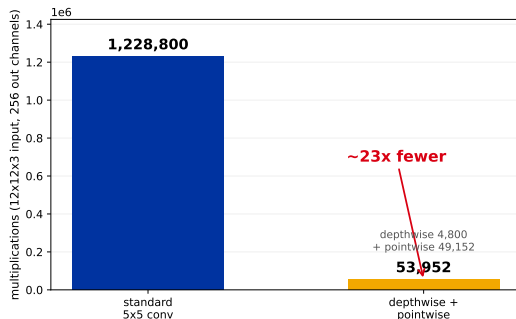
Standard:

$$256 \cdot \underbrace{(5 \cdot 5 \cdot 3)}_{\text{per stop}} \cdot \underbrace{(8 \cdot 8)}_{\text{stops}} = 1,228,800$$

Depthwise + pointwise:

$$\underbrace{3 \cdot 25 \cdot 64}_{4,800} + \underbrace{256 \cdot 3 \cdot 64}_{49,152} = 53,952$$

$\sim 23\times$ fewer multiplications for the same output shape. This is why a pretrained detector runs *on* the sorting line, not in a cloud.



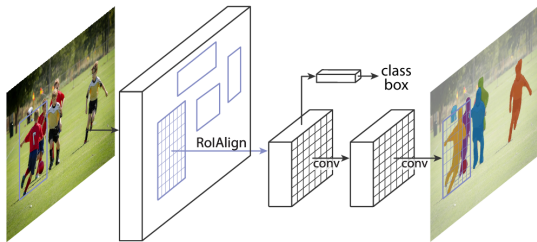
Segmentation

Every pixel gets an answer.

Semantic vs instance

Segmentation = **per-pixel classification**.

- **Semantic**: every pixel gets a class label - all our pomegranates merge into *one* fruit mask (task-zoo panel 4).
- **Instance**: each fruit gets its **own** mask. **Mask R-CNN** = Faster R-CNN + a small mask head per region (He et al., 2017).
- **Panoptic**: both at once - every pixel labeled *and* grouped into instances.



Mask R-CNN: detection boxes plus a per-region mask branch (He et al., 2017).

And from the L17 epilogue: **SAM** made segmentation a *promptable* foundation-model capability - click an object, get its mask.

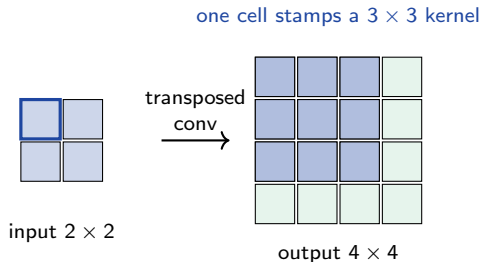
Transposed convolution: the output must be an image

Convs and pools **shrink** maps - but a per-pixel answer needs **full resolution** back: a learnable way *up*.

A **transposed convolution** upsamples: each input cell *stamps* the kernel onto the output, overlaps add.

Matrix view, one line: a conv is a sparse multiply $\mathbf{y} = \mathbf{K}\mathbf{z}$; the transposed conv uses $\mathbf{K}^\top$'s *sparsity pattern* to go back up - with its own learned weights. Needed whenever the **output is an image**.

Segmentation needs it now; *generators* reuse it at the end of this lecture.



Watch-out: checkerboard artifacts. Uneven kernel overlaps tile the output with a checker pattern (L16's 1D toy: the middle cell got two stamps). Fixes: kernel size divisible by the stride, or upsample-then-convolve (Odena et al., 2016).

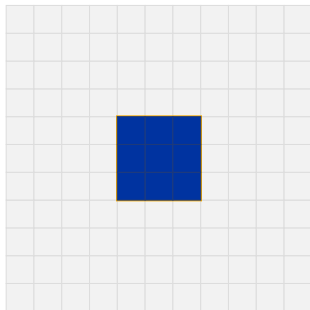
Dilated convolution: context without losing resolution

Insert **gaps** into the kernel: a 3×3 kernel with dilation d taps cells d apart. (**Click to widen.**)

- ▶ Same 9 weights, wider view - no pooling, no lost pixels.
- ▶ L17 grew the receptive field by *stacking*; dilation grows it **per layer**.
- ▶ Beyond images: **WaveNet** audio and TCN time-series models stack dilated 1D convs.

Segmentation's dilemma solved: see the **whole scene** while keeping **per-pixel** detail.

dilation $d = 1$: the same 9 weights span 3×3



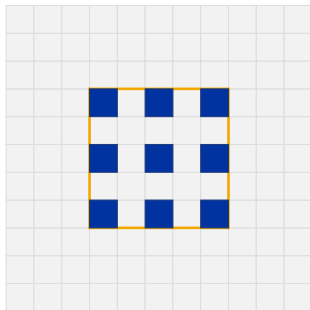
Dilated convolution: context without losing resolution

Insert **gaps** into the kernel: a 3×3 kernel with dilation d taps cells d apart. (**Click to widen.**)

- ▶ Same 9 weights, wider view - no pooling, no lost pixels.
- ▶ L17 grew the receptive field by *stacking*; dilation grows it **per layer**.
- ▶ Beyond images: **WaveNet** audio and TCN time-series models stack dilated 1D convs.

Segmentation's dilemma solved: see the **whole scene** while keeping **per-pixel** detail.

dilation $d = 2$: the same 9 weights span 5×5



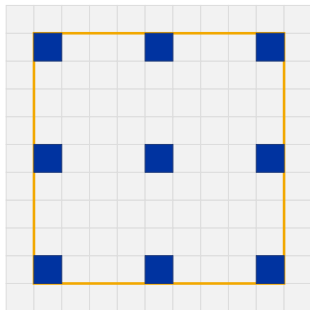
Dilated convolution: context without losing resolution

Insert **gaps** into the kernel: a 3×3 kernel with dilation d taps cells d apart. (**Click to widen.**)

- ▶ Same 9 weights, wider view - no pooling, no lost pixels.
- ▶ L17 grew the receptive field by *stacking*; dilation grows it **per layer**.
- ▶ Beyond images: **WaveNet** audio and TCN time-series models stack dilated 1D convs.

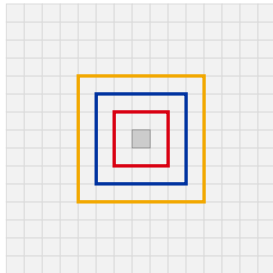
Segmentation's dilemma solved: see the **whole scene** while keeping **per-pixel** detail.

dilation $d = 4$: the same 9 weights span 9×9



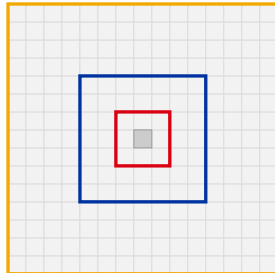
Stack dilations and the field explodes

3 standard 3x3 layers
receptive field grows linearly



— layer 1 ($d=1$): 3×3
— layer 2 ($d=1$): 5×5
— layer 3 ($d=1$): 7×7

3 dilated 3x3 layers ($d = 1, 2, 4$)
receptive field grows exponentially



— layer 1 ($d=1$): 3×3
— layer 2 ($d=2$): 7×7
— layer 3 ($d=4$): 15×15

same cost either way: 9 weights per layer

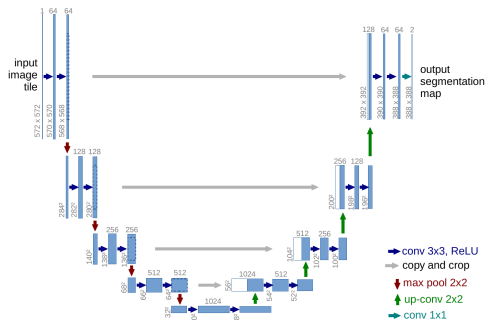
Doubling the dilation each layer ($d = 1, 2, 4$) grows the receptive field **exponentially**: 15×15 instead of 7×7 , for the same nine weights per layer (compare L17's receptive-field frame).

U-Net: the segmentation workhorse

Both tools on the table - the architecture writes itself:

- **Down:** convs + pooling compress the image into context.
- **Up:** transposed convs climb back to full resolution.
- **Skips:** early, sharp maps are *concatenated* across - detail the bottleneck destroyed.

L17's skips again - there they rescued **optimization**, here they carry **resolution**. Dilated convs often grow context mid-net.

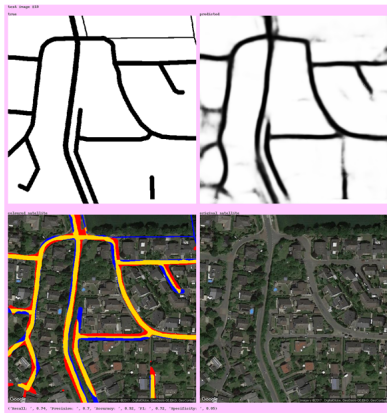


U-Net in action: roads from the sky

A U-Net trained to segment **roads in satellite imagery**:

- ▶ input: an RGB tile $[512, 512, 3]$;
- ▶ output: a road probability **per pixel** $[512, 512, 1]$;
- ▶ loss: [11]'s log-loss, applied *once per pixel* and summed.

Top row: truth vs prediction. Bottom left: yellow = correct road pixels, blue = missed, red = false alarms - [12]'s confusion matrix, *painted onto the map*.



Source: LMU I2DL (slds-lmu/lecture_i2dl), CC BY 4.0.

The sorting line, completed

Cash in the running example - then zoom out on the whole chapter.

One scenario, four output structures

The pomegranate line, end to end:

Question on the line	Task	Tool
"what grade is this fruit?"	classification	L16's CNN, built via L18 transfer
"is it grading fruit or belt?"	attribution	L18's Grad-CAM audit
"how many fruit, and where?"	detection	a fine-tuned YOLO
"where exactly is the bruise?"	segmentation	a small U-Net for the cutter

Swap the nouns and it is the same story everywhere: roads on satellite tiles, tumors on MRI slices, cracks on factory parts.

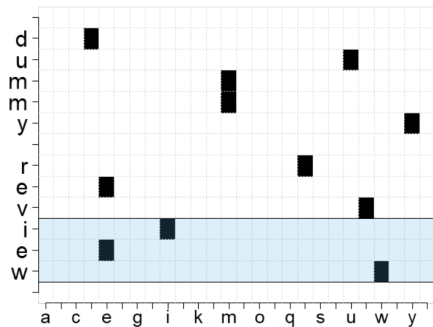
One backbone vocabulary - conv, pool, skip, upsample - and the **output structure** turns it into whichever tool the problem needs.

Same idea, other shapes: 1D and 3D

Nothing about convolution says "image". The window slides over whatever grid the data lives on:

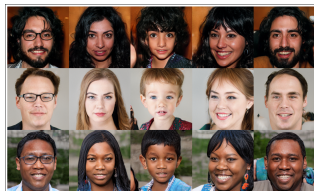
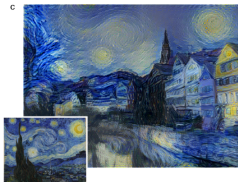
- ▶ **1D** - sequences: accelerometer traces for activity recognition, audio, character-level text (right). L16 *opened* with a 1D kernel sliding over a sequence - that was the real thing, not a warm-up.
- ▶ **2D** - images: this whole chapter.
- ▶ **3D** - volumes: video (stacked frames), MRI and CT scans - the kernel slides in x , y and z .

Convolution is a **structural prior** - "nearby means related, patterns repeat" - not an image trick. Wherever that assumption holds, the same layer works.



A movie review as a **1D signal**: each row is a character of "dummy review", one-hot across the alphabet; a 1D kernel (blue) slides down it - a **character-level text-CNN**.

CNNs can also paint



Neural style transfer: repaint a photo in a painting's style by matching CNN *features* - content = deep-layer activations, style = feature correlations. The features come from **VGG** (L17!).

GANs: two networks duel - a generator forges images, a discriminator calls fakes, both improve until the fakes pass. None of these people exist. The generator upsamples noise into pixels with **transposed convolutions** - today's tool.

Honest footnote: for image *generation*, **diffusion** has largely taken over (L17 epilogue) - GANs live on in upscaling and faces. Mechanics belong to the diffusion chapter (L27-L31).

Gatys et al. 2015; Goodfellow et al. 2014; StyleGAN faces: Karras et al. 2019.

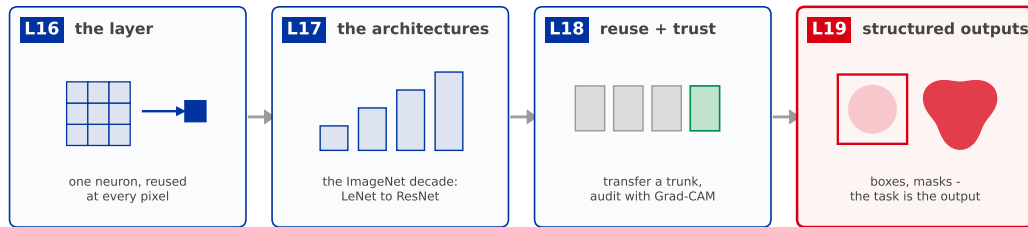
The inductive-bias ledger

What this chapter really bought us - stated honestly:

- ▶ CNNs **assume** locality and translation equivariance (L16's two wishes: "nearby means related, a detector useful here is useful there").
- ▶ When the assumption is *true* - images, audio, volumes - it acts as **regularization by architecture**: win with *less data*.
- ▶ When it is *false*, it costs you: **tabular data** still belongs to trees and boosting (ch4, and [40]'s tabular caveat), and images where **global context** dominates strain the local prior.

There is no universally best architecture - only assumptions that **match or mismatch** the data. Choosing the bias *is* the modeling decision.

The chapter in one strip



One layer → architectures → reuse and trust → structured outputs.
You can now read, adapt, audit and deploy the networks that do most of the world's looking.

Hand-off: Vision Transformers

The L17 epilogue's loose thread, tied:

- ▶ **ViT** cuts the image into patches and feeds them to a transformer as a *sequence* - **no convolution at all** (Dosovitskiy et al., 2021).
- ▶ It **drops the convolutional prior** - and buys it back with **scale**: with enough pretraining data, attention *learns* locality instead of assuming it.
- ▶ Small data? The CNN prior still wins. Web-scale data? Attention takes the frontier. The ledger from two slides ago, live in 2026.

Attention, transformers and the GenAI stack get their own chapters - vision handed the baton; we follow it there next.

Recap

- ▶ The **output structure defines the task**: label $\rightarrow$ box $\rightarrow$ box per object $\rightarrow$ class per pixel.
- ▶ Detection vocabulary: **IoU** (our worked 1/3), **NMS** (keep the best, drop overlapping neighbors), **mAP** (= L12 over boxes).
- ▶ Two families: two-stage (Faster R-CNN) vs one-stage (**YOLO**: 7×7 grid, all boxes in one pass).
- ▶ **Separable convs** make it fit on a belt camera ($\sim 23\times$ fewer multiplications).
- ▶ Segmentation: semantic vs instance; **U-Net** = down, up (**transposed convs**), skips - with **dilation** for cheap context.
- ▶ Convolution is a **prior with a domain**: 1D/2D/3D where locality holds, trees on tabular, attention at web scale.

Next: HW4 - run a pretrained YOLO on *your* photos and turn the NMS IoU knob; swap a dilated conv into the L16 CNN. Then a new chapter: networks with **memory** - RNNs (L20).